Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307P - Compiler Design Lab

Report Submission Guidelines

Project Title: OptiGuard - An Adaptive Compiler Optimization Framework with Degradation Detection and Safe Rollback

Code Link: https://github.com/rithika20252024/optiguard

Team Member(s): Max Three members

S. No	Register No	Name	Contributions
1	24BCT0156	**Aesha Singh**	Common Subexpression Elimination (value numbering) and strength reduction; IR feature extractor; cost function J(IR) and Delta J degradation detection; adaptive pass scheduler; literature survey (papers 1-8) and novelty section
2	24BCE2984	Rithika	Lexer, recursive-descent parser, TAC/CFG generation; Constant Folding, Algebraic Simplification, Dead-Code Elimination; stack-based snapshot, rollback engine and tabu list; benchmark evaluation; literature survey (papers 9-15) and problem statement

Problem Identification & Objectives - 2M

Problem Statement

Production compilers such as GCC and LLVM/Clang run a fixed, preset order of optimization passes inside levels like -O1, -O2 and -O3. Compiler transformations are non-commutative, so no single ordering suits every program, and with N passes there are N! possible orderings. Each pass is designed assuming a local benefit, yet in practice an apparently beneficial pass can degrade the IR and yield worse machine code. Three typical causes are register-pressure explosion (aggressive CSE or LICM extends live ranges and forces memory spills), code bloat and instruction-cache thrashing (aggressive inlining and unrolling), and phase destabilization (an early pass destroys the canonical patterns that later passes need).

The gap is that passes run speculatively and irreversibly. Existing compilers and autotuners search for good orderings offline or at flag level, but none measure the IR between passes, and none can atomically undo a harmful pass. This project addresses the need for a compiler that treats every optimization pass as a transaction: measure it, keep it if it helps, and roll it back if it hurts.

Objectives (Minimum 3)

- **Frontend and IR pipeline:** * Frontend and IR pipeline: build a lexer and recursive-descent parser for a procedural C-subset that lowers programs to observable Three-Address Code (TAC) and a Control Flow Graph (CFG).
- **Optimization suite:** * Optimization suite: implement Constant Folding and Propagation, Algebraic Simplification and Strength Reduction, Dead-Code Elimination, and CSE via Value Numbering, each reporting the instructions it changes.
- **Multi-objective cost function:** * Multi-objective cost function: define J(IR) = w1*N_instructions + w2*R_live_ranges + w3*C_alu score every intermediate IR state.

- **Degradation detection:** * Degradation detection: compute Delta J = J(IR_new) - J(IR_prev) after every pass, and raise a degradation interrupt when Delta J >= 0 or a register-pressure threshold is breached.
- **Transactional rollback:** * Transactional rollback: use stack-based snapshots to restore the IR exactly to the last checkpoint and record the offending pass on an IR-state tabu list.
- **Adaptive pass scheduler:** * Adaptive pass scheduler: choose the next pass from real-time IR structural features instead of a hardcoded list, and evaluate it against a fixed-order baseline on a benchmark set.

Objectives are kept specific and measurable: instruction count, live-range count and ALU-cycle cost before versus after optimization, number of degradations detected and rolled back, and final J(IR) against the fixed-order baseline.

Literature Survey - 2M

The table below summarizes 15 relevant papers, tools and existing systems.

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
1	Lattner & Adve, 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation (CGO)	**Typed SSA-based IR with a modular pass manager**	Pass pipelines are composed from reusable passes over a common IR	Fixed -O levels; no per-pass measurement of degradation and no undo
2	Bodin, Kisuki, Knijnenburg, O'Boyle & Rohou, 1998. Iterative Compilation in a Non-Linear Optimisation Space	Repeatedly compiles and runs a program with different optimization parameters	Search over parameters beats static settings on the target	Needs many full compile-and-run cycles; slow and machine-specific
3	Cooper, Schielke & Subramanian, 1999. Optimizing for Reduced Code Space using Genetic Algorithms (LCTES)	Genetic algorithm evolves pass sequences for code size	Evolved sequences outperform the fixed default ordering	Offline search over hours; a bad sequence is only scored poorly, the culprit pass is never isolated
4	Kulkarni, Hines, Hiser, Whalley, Davidson & Jones, 2004. Fast Searches for Effective Optimization Phase Sequences (PLDI)	Search over phase sequences with pruning of redundant sequences	Cuts search cost while finding good sequences	Requires execution or simulation for feedback; no monitoring between passes
5	Pan & Eigenmann, 2006. Fast and Effective Orchestration of Compiler Optimizations for Automatic Performance Tuning (CGO)	Iterative elimination of harmful optimization flags using runtime profiles	Selects flag combinations that improve measured performance	Needs runtime profiling on hardware; decisions are at flag level, not IR level
6	Agakov et al., 2006. Using Machine Learning to Focus Iterative Optimization (CGO)	Learns from past programs to focus the search on promising transformations	Reduces the number of search iterations needed	Model quality depends on training data; no safety net if a prediction is wrong
7	Fursin et al., 2011. Milepost GCC: Machine Learning Enabled Self-Tuning Compiler (IJPP)	56 static program features feed supervised learning that predicts optimization flags	Automatically tunes GCC for unseen programs	Macro-flag granularity; black-box prediction with no internal revert
8	Ansel et al., 2014. OpenTuner: An Extensible Framework for Program Autotuning (PACT)	Ensemble of search techniques for autotuning arbitrary parameters	General framework reusable across domains	Domain-agnostic and measurement-driven; knows nothing about IR state
9	Ashouri et al., 2017. MiCOMP: Mitigating the Compiler Phase-Ordering Problem Using Optimization Sub-Sequences and Machine Learning (ACM TACO)	Clusters optimization sub-sequences and predicts the best with ML	Improves over standard -O levels on benchmark suites	Predicts sequences ahead of time; no in-flight detection of a harmful pass
10	Ashouri, Killian, Cavazos, Palermo & Silvano, 2018. A Survey on Compiler Autotuning Using Machine Learning (ACM CSUR)	Survey of two decades of ML-based compiler autotuning	Classifies selection and phase-ordering approaches	Highlights degradation and robustness as open problems and calls for self-stabilizing compilers